

# Customer Churn Prediction & LTV Engine

Week 4 • Day 6

## GitHub Cleanup, Resume Points & Interview Prep

Polish your GitHub profile, write resume bullet points, and prepare for interviews

### ■ Day Objectives

- ◆ Clean up GitHub repository (proper gitignore, folder structure, tags)
- ◆ Write 5 resume bullet points from this project
- ◆ Prepare answers to 20 common data analytics interview questions
- ◆ Create a project presentation slide outline
- ◆ Record a 2-minute project demo video script

## ■ 1. GitHub Repository Cleanup

### ■ Final GitHub cleanup commands

```
# Create a release tag
git tag -a v1.0.0 -m 'Production-ready churn prediction system'
git push origin v1.0.0

# Verify .gitignore excludes sensitive files
cat .gitignore
# Should include: .env, __pycache__, *.pkl, data/raw/

# Count total commits
git log --oneline | wc -l
```

```
# Should be >= 28
```

## ■ 2. Resume Bullet Points

**How to write impact-driven resume bullets:** [Action Verb] + [What you built] + [Technology used] + [Measurable result]

- Built a production-grade Customer Churn Prediction system using XGBoost and Python, achieving AUC-ROC of 0.86 on 7,043 customer records
- Engineered 15+ predictive features from raw telecom data, improving model recall for churned customers by 18% over baseline Logistic Regression
- Deployed a FastAPI REST API serving real-time churn predictions and LTV estimates with <200ms response time, containerised with Docker
- Designed a PostgreSQL data warehouse and ETL pipeline using SQLAlchemy, processing and cleaning 7,043 customer records with automated quality checks
- Created an interactive Metabase analytics dashboard with 8 KPI charts for business stakeholders, revealing that month-to-month contract customers churn at 42.7% — 15× higher than two-year contract holders

## ■ 3. Interview Q&A;

### Technical Interview Questions & Model Answers

---

#### Q: Why did you choose XGBoost over Logistic Regression?

A: Logistic Regression assumes a linear relationship between features and log-odds of churn, but churn drivers interact non-linearly (e.g., a new customer with fiber optic AND electronic check payment is disproportionately risky). XGBoost captures these interactions through tree splitting. Our empirical results confirmed: XGBoost AUC 0.86 vs LR AUC ~0.79.

#### Q: How did you handle class imbalance?

A: The dataset has 73.5% No-churn / 26.5% Churn imbalance. For Logistic Regression and Random Forest, I used `class_weight='balanced'` which inversely weights classes by frequency. For XGBoost, `scale_pos_weight=2.77` (5174/1869). I evaluated with F1-score and AUC-ROC rather than accuracy, since a model predicting 'No Churn' always gets 73.5% accuracy.

#### Q: What is SHAP and why is it important for business?

A: SHAP (SHapley Additive exPlanations) assigns each feature a contribution value for each individual prediction. For business, this transforms the ML model from a black box into an explainable system. Instead of just saying 'Customer X will churn', we can say 'Customer X will churn because: (1) they are on a month-to-month contract (+0.32 SHAP), (2) they have no online security (-0.15 absent), (3) their tenure is only 2 months (+0.28 SHAP)'. This drives targeted retention actions.

#### Q: What is AUC-ROC?

A: The ROC curve plots True Positive Rate against False Positive Rate at every classification threshold. The AUC (Area Under the Curve) is a single number summarising this: 0.5 = random guessing, 1.0 =

perfect classifier. Our XGBoost AUC of 0.86 means: randomly pick a churner and a non-churner — there is an 86% probability the model ranks the churner higher.

**Q: How would you improve this model further?**

A: Several directions: (1) Add temporal features — how have charges trended over the last 3 months? (2) Use SMOTE oversampling instead of class weights. (3) Add external data — customer support call logs, NPS scores. (4) Implement model monitoring to detect drift as new data arrives. (5) A/B test retention campaigns using the model's recommendations.

## ■ 4. Commit

■ **Week4-Day6 Commit**

```
git add .
git commit -m "Week4-Day6: GitHub cleanup, v1.0.0 tag, resume points, interview prep"
git push origin main
git push origin v1.0.0
```

■ **Estimated Time: 5 – 6 hours**

— End of Week 4 Day 6 — Zaalima Development Confidential —